

Assignment 2B — RAG Pipeline

Overview

Build a small, complete RAG system on your own domain corpus: choose how to chunk the documents, compare ways of searching them, improve the results with reranking, then generate grounded answers and check them. This covers the core of a RAG system, not a full production deployment. Use one notebook for the whole assignment.

Reuse from Assignment 1: your cleaned domain .txt files as the document collection, 10 queries from your Assignment 1 evaluation set with their expected answers, and your fine-tuned model as the answer generator. No new data collection is needed.

Part	What you do	Marks
A	Build dense search and choose a chunk size	5
B	Compare dense, keyword (BM25) and hybrid search	4
C	Rerank search results	4
D	Generate grounded answers and check them	7
Total		20

Each numbered step shows its marks in brackets, so every step is graded on its own.

Part A — Dense search and chunk size (5 marks)

Before you start: for each of your 10 queries, mark the answer span, the sentence or short passage in your corpus that answers it. A retrieved chunk is a **hit** if it contains that span. You will reuse these spans in Parts A, B and C.

Also note your embedding model's maximum input length, for example 256 tokens for all-MiniLM-L6-v2, and count tokens with that model's tokenizer.

A1. Build dense search (2 marks)

1. Write one reusable function that embeds a set of chunks, builds a FAISS IndexFlatIP index (or another vector index) and returns the top k chunks for a query. **(1 mark)**
2. Report the embedding model name, vector size, similarity measure, batch size, and how long the index takes to build for your final chunk set. **(1 mark)**

A2. Choose a chunk size (3 marks)

Any text beyond the embedding model's limit is cut off when embedded. If a size exceeds the limit, either switch to a model with a longer limit or replace that size with one that fits, and say which you did.

1. Chunk your corpus with fixed-size chunking at 100 and 400 tokens, with no overlap. Run your dense search from A1 on the 10 queries for each size and fill this table. **(2 marks)**

2.

Chunk size (tokens)	Total chunks (1M)	Hit rate@3 (%) (	Tokens sent per query (top 3)
100			
500			

Hit rate@3 is the share of the 10 queries where one of the top 3 chunks contains the answer span. Tokens sent per query is the combined size of the top 3 chunks, averaged over the queries.

- In two or three sentences, name the chunk size you will use in Parts B to D and why. Consider hit rate, tokens per query, and how much context your Assignment 1 model can handle. **(1 mark)**

Part B — Keyword and hybrid search (4 marks)

Use the chunks you chose in Part A. Compare your dense search from Part A with keyword search and a hybrid of the two, on the same 10 queries, returning the top 5 chunks each time. Each search method fills one row of the table below.

- Dense:** run your dense search from Part A on the 10 queries and fill the Dense row. **(1 mark)**
- BM25:** build a BM25 index, retrieve the top 5 for each query, and fill the BM25 row. **(1 mark)**
- Hybrid:** combine the dense and BM25 results with Reciprocal Rank Fusion (RRF), keep the top 5, and fill the Hybrid row. **(1 mark)**

Each chunk's score is:

$$\text{RRF}(d) = 1 / (60 + \text{rank in dense list}) + 1 / (60 + \text{rank in BM25 list})$$

Fuse the top 20 from each method, not just the top 5, so a chunk ranked fairly well by both can still rise to the top. Ranks start at 1, and a chunk missing from one list gets nothing from that list. Fill this table for all three methods.

Method	Average query latency (ms)	Hit rate@1 (%)	Hit rate@3 (%)
Dense			
BM25			
Hybrid (RRF)			

Compute the columns like this:

Metric	How to compute it
Query latency (ms)	Time to return the top 5 for one query, averaged over the 10 queries; skip one warm-up run first
Hit rate@1 (%)	Share of the 10 queries whose first result contains the answer span
Hit rate@3 (%)	Share of the 10 queries where at least one of the top 3 results contains the answer span

4. In two or three sentences, say which method works best for your queries and why, using one query as an example. (1 mark)

Part C — Reranking (4 marks)

1. **Rerank:** take your best method from Part B and retrieve the top 10 for each query. Re-score them with a cross-encoder, for example cross-encoder/ms-marco-MiniLM-L-6-v2, and keep the top 3. (1 mark)
2. **Compare rankings:** for each query, show the top 3 before and after reranking, and report the rank change rate. (1 mark)
3. **Measure:** report the reranking time per query and the hit rate@1 after reranking, and fill this table. (1 mark)

Measure	Value
Reranking time per query (ms)	
Rank change rate (%)	
Hit rate@1 before reranking (from Part B)	
Hit rate@1 after reranking	

Rank change rate is the share of the 10 queries whose first result changes after reranking.

4. In two or three sentences, say whether reranking helped in your domain, using the hit rates before and after and the time it added. (1 mark)

Part D — Generate and check answers (7 marks)

D1. Generate grounded answers (3 marks)

1. Write a prompt that tells the model to answer only from the chunks, cite the chunk numbers it used, for example [2], and reply "Not found in the documents" when the chunks do not contain the answer. Show the prompt. (1 mark)
2. For each of the 10 queries, give your Assignment 1 model the top 3 chunks after reranking, each labelled with a number, and show all 10 answers. (1 mark)
3. Add one query whose answer is not in your corpus, and show that the model replies that it cannot find it. (1 mark)

If your Assignment 1 model ignores the chunks or never says "Not found", you may switch to an instruct model such as Llama 3.1 8B Instruct or Qwen2.5 7B Instruct. Name the model you used and show one example of the problem you saw.

D2. Check the answers with an LLM judge (4 marks)

An LLM judge is a second LLM that reads an answer and decides whether it meets criteria you define, like an examiner with a marking scheme. Use a judge model different from your generator, for example Llama 3.1 8B Instruct or Qwen2.5 7B Instruct, and name it.

Hints for a good judge prompt:

- Give the judge everything it needs to decide: the question, the expected answer, the retrieved chunks and your model's answer.
- Ask one clear question per check, not "Is this a good answer?"
- Define each label exactly. For example, say what makes an answer "partly correct" rather than "wrong."
- Say how to treat a "Not found in the documents" reply.
- Ask for a one-sentence reason with each verdict, and a quote of any unsupported statement.
- Fix the output format, for example CORRECT: ... and SUPPORTED: ... on separate lines, so you can read the results with code.
- Set the temperature to 0, so the judge gives the same verdict every time.

1. **Write and run the judge:** write one judge prompt that checks two things for each answer:
 - Correct: correct, partly correct or wrong, compared with the expected answer from your Assignment 1 evaluation set.
 - Supported: yes or no, whether every statement can be found in the answer's 3 chunks.

Show your prompt, run it on all 10 answers, show the results in one table with a row per query (query, correct, supported, reason), and fill the summary below. (1 mark)

Check	Count out of 10
Correct	
Partly correct	
Wrong	
Supported by the chunks	

2. **Spot-check the judge:** read 3 of the answers and their chunks yourself. Say how many of the judge's verdicts you agree with, and explain one disagreement if there is one. (1 mark)

3. **Find the cause:** pick one answer that is wrong or not supported, and show its 3 chunks. Say whether retrieval or generation caused the problem: was the needed information in the chunks or not? (1 mark)
4. **Suggest a fix:** propose one change that would fix it, for example a different chunk size, search method, reranking or prompt wording, and explain why it would help. (1 mark)

Submission checklist

Files

- Notebook (.ipynb) with all outputs visible
- HTML export of the same notebook

Part A

- Answer span marked for each of the 10 queries
- A1: embedding model name, vector size, similarity measure, batch size, index build time
- A2: chunk size table (100 and 400 tokens) and chosen size with reason

Part B

- Search comparison table (Dense, BM25, Hybrid)
- Best method and why, with an example query

Part C

- Top 3 before and after reranking for each query
- Reranking table (rank change rate, time per query, hit rate@1 before and after)
- Whether reranking helped, weighing accuracy against time

Part D

- D1: generation prompt, all 10 answers, and the "Not found" query with its answer
- D1: generator model used, plus one example of the problem if you switched from your Assignment 1 model
- D2: judge model name and judge prompt
- D2: judge results table (per query) and summary counts
- D2: spot-check of 3 verdicts, with one disagreement explained if any
- D2: one failed answer with its 3 chunks, its cause, and a suggested fix